Unil.
Université de Lausanne
HEC Lausanne

Session 6: Real data, real errors

Reading and writing files, handling bad data, and what to do when a program breaks.

Data Science and Advanced Programming · UNIL · 2026

Data Science and Advanced Programming · UNIL · 2026

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

data from a file

results to a file

```
$ python report.py
not run yet
```

Every number so far was typed into the script

Sessions 2–5 wrote the ledger **inside** the code: `shares = [10, 4, 2]`. Real data never arrives like that. It arrives in **files**: exported from a bank, downloaded from the web, sent by a colleague.

And real files are messy: a missing value, an odd character, a row that breaks the rules.

Today: read a file, turn its text into numbers, survive the bad rows, write a report, and **find the problem** when something breaks.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
not run yet
```

Today: one script, from file to report

1. Files and paths, from Python
2. Text in, numbers out: reading a CSV
3. Errors as tools: `try`, `except`, `raise`, `assert`
4. Tracebacks through functions and libraries
5. Debugging, as a method
6. The complete program

Session 1 promised a program that does **load → compute → write**. This is it.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
not run yet
```

The data: `ledger.csv`

The file next to the script, as plain text:

```
ticker,name,shares,price
AAPL,Apple,10,190.50
NESN,Nestlé,4,102.30
VOO,Vanguard,2,512.00
MSFT,Microsoft,,410.00
```

One **header** line, then one line per position, fields separated by commas. Look at the last line: MSFT's **shares are missing**. Real files look like this.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
not run yet
```

Read it cold

Two minutes, on your own.

- Two functions, `load` and `write_report`. What does each one take, and return?
- Find the words `with`, `try`, `except`, `assert`. Guess what each does.
- **Guess the output.** What happens to MSFT?

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number        ← the bad row
3 positions, total 3338.2
Apple      1,905.00             ← report.txt, read back
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

What it prints

You do not need to run anything now. Bottom left is the output of one run.

- The first line: MSFT was **skipped**, and the program **said so**.
- Then a summary: 3 positions, total 3338.2, session 3's number once more.
- The last lines are the **file the program wrote**, `report.txt`, read back and printed.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

REMINDER

Files and paths (session 1)

- A file has **contents**, a **name** (with an extension: `.csv`, `.py`) and a **place** in the folder tree.
- A **path** is its address. `ledger.csv` is a **relative** path: "in the folder where the program stands".
- The program stands in its **working directory**. Run it from the wrong folder, and `ledger.csv` is not there: session 1's `FileNotFoundError`.

Today, Python itself opens, reads and writes those files.

PART 1

Files and paths, from Python

Open a file, read it line by line, write a new one.

report.py

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             try:
12                 rows.append((name, int(shares), float(price)))
13             except ValueError:
14                 print(f"skipped {ticker}: bad number")
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

open, and name it f

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

`with open(...) as f:`

line 6	`with open(path, encoding="utf-8") as f:`
open(path, …)	connect to the file on disk
encoding=	how its bytes become text (later)
as f	f: a file object, to read from
with …:	**close the file when the block ends**

The indented block (lines 7–13) is the time the file is **open**. When it ends, even because of an error, `with` closes the file for you.

Always open files with `with`. You will see it in every good script, and in most AI-generated ones.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

one line: the header

then each remaining line

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

A text file is a sequence of lines

```
ticker,name,shares,price\n      ← line 7: f.readline()
AAPL,Apple,10,190.50\n          ← line 8: for line in f: …
NESN,Nestlé,4,102.30\n
VOO,Vanguard,2,512.00\n
MSFT,Microsoft,,410.00\n
```

A file object reads **one line at a time**. `f.readline()` takes the next line: here the header, which we keep aside. `for line in f:` then goes through **every remaining line**, like a list.

Each line ends with an invisible character: `\n`, "new line".

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             try:
12                 rows.append((name, int(shares), float(price)))
13             except ValueError:
14                 print(f"skipped {ticker}: bad number")
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total == 3338.2, "an empty report?"
29 print(repr(open("ledger.csv").readline()))
30 print(Path("report.txt").read_text())
```

show what is really there

```
$ python report.py
skipped MSFT: bad number
'ticker,name,shares,price\n'
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

the invisible \n

The invisible `\n`

Line 29 now prints the header line with `repr()`: the string **as Python sees it**, quotes and all. At the end: `\n`.

What is `\n`? A text file has no "lines" in it: it is one long row of characters, and the end of each line is marked by an **invisible character**, the **newline**. Python writes it as a backslash and an `n`: **two symbols on screen, one character** (`len("\n")` is 1). The backslash says "the next letter is special"; `\t` is a tab.

`print` hides it: it just moves to a new line. `repr` shows it. When text behaves strangely, **`repr` is your microscope**.

That `\n` is why line 9 calls **`.strip()`**: it removes spaces and `\n` from both ends, before we split the line.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Three ways to read a file

You write	Gives	Good for
`for line in f:`	one line at a time	big files, line-based formats (line 8)
`f.readline()`	the next line only	the header (line 7)
`f.readlines()`	a list of all lines, each with its `\n`	small files
`Path(...).read_text()`	the whole file as one string	small files (line 30)

All four give **text**. A file object also remembers **where it is**: after `readline()`, the loop starts at the second line. A file read to the end gives nothing more.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            rows.append((name, int(shares), float(price)))
        except ValueError:
            print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

strip, then split

unpack 4 fields

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

From a line to four fields

For the NESN line:

line `"NESN,Nestlé,4,102.30\n"`

↓ .strip(): remove the \n

`"NESN,Nestlé,4,102.30"`

↓ .split(","): cut at each comma (session 3)

parts `["NESN", "Nestlé", "4", "102.30"]`

↓ unpack (session 4): one name per field

line 10 **`ticker="NESN"`** **`name="Nestlé"`**

`shares="4"` **`price="102.30"`**

Every field is still **text**. `"4"` is not 4 (session 2). Converting comes next.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

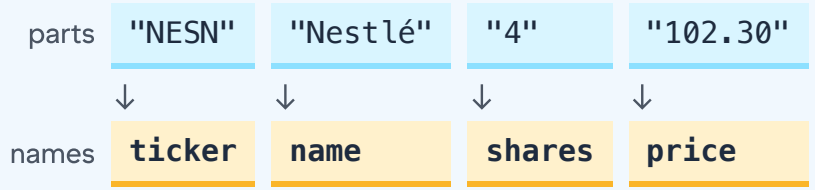
4 names ← 4 values

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

REMINDER

Unpacking (session 4)

Several names on the left of `=`, a list or tuple on the right: each name gets **one value, in order**.



The counts must **match**: 4 names need exactly 4 values, or Python stops with a `ValueError` (we will meet it in Part 2). Session 4 used it for `v, w = value_and_weight(...)`: same idea, a function's tuple instead of a split line.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

w: write

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Writing a file

Line 19 opens `report.txt` in mode `"w"`, for writing. `out.write(text)` adds text to it.

- `write` does **not** add a new line by itself: the `\n` at the end of each f-string does.
- The f-string formats (session 2) line the columns up: `:<9` pads the name, `:>10,.2f` right-aligns the value.
- When the `with` block ends, the file is closed and saved.

Output lines 3–6 are that file, read back by line 30.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

erases the old file

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

⚠ WARNING

`"w"` erases, `"a"` appends

Mode	Opens for	If the file exists
`"r"` (the default)	reading	reads it
`"w"`	**writing**	**empties it first, without asking**
`"a"`	appending	adds at the end

`open("ledger.csv", "w")` would **wipe your data** the moment it runs: no warning, no undo. Keep inputs and outputs as **different files**, as this script does.

With `"a"` instead, every run would **add** a second report below the first: a file that grows each time you press Run.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="latin-1") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             try:
12                 rows.append((name, int(shares), float(price)))
13             except ValueError:
14                 print(f"skipped {ticker}: bad number")
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

the wrong encoding

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
NestlÃ©       409.20
Vanguard   1,024.00
total      3,338.20
```

é became Ã©

Encoding: how bytes become text

A file stores **bytes** (session 1). The **encoding** is the agreement that turns them into characters. `ledger.csv` was saved as **UTF-8**, the modern standard, where `é` takes two bytes.

Line 6 now reads it as `latin-1`, an older agreement. Each of `é`'s two bytes becomes a character of its own: **`NestlÃ©`**. No error, just garbled text.

When you see `Ã©`, `Ã¨` or `â€™` in data, suspect the encoding. Always write `encoding="utf-8"`.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

paths as objects

a Path, not just text

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

`pathlib`: paths as objects

`Path("ledger.csv")` is a **Path object** (session 4), not a string, with methods for everything you do with files.

You write	Gives
`Path("ledger.csv").exists()`	`True` or `False`
`Path("data") / "ledger.csv"`	the path `data/ledger.csv`, on any system
`Path("report.txt").read_text()`	the whole file as one string (line 30)
`Path("ledger.csv").suffix`	`".csv"`

`/` joins folder names safely on any system.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

+ GOOD TO KNOW

Where is "here"?

`Path("ledger.csv")` is **relative**: Python looks in the **working directory**, the folder the program was started from (session 1). `Path.cwd()` tells you which folder that is.

Two habits that avoid most "file not found" errors:

- run your script **from its own folder**;
- or build the path from the script's own location: `Path(__file__).parent / "ledger.csv"`, which works wherever you start the program from.

`__file__` is a dunder (session 4): the path of the script itself.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             try:
12                 rows.append((name, int(shares), float(price)))
13             except ValueError:
14                 print(f"skipped {ticker}: bad number")
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledgr.csv"))   # a typo
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

```
$ python report.py
Traceback (most recent call last):
  File "report.py", line 26, in <module>
    rows = load(Path("ledgr.csv"))
           ^^^^^^^^^^^^^^^^^^^^^^
  File "report.py", line 6, in load
    with open(path, encoding="utf-8") as f:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'ledgr.csv'
```

`FileNotFoundError`, again (session 1)

Line 26 now asks for `ledgr.csv`. The last line says exactly what session 1 said: `No such file or directory: 'ledgr.csv'`.

Two usual causes:

- the **name** is wrong, as here;
- the name is right, but the program stands in **another folder** (session 1).

The traceback has **two** frames this time. Part 4 reads it properly.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

+ GOOD TO KNOW

Why `with`: files must be closed

An open file holds on to resources, and what you **write** may sit in memory until the file is closed. A program that crashes with a file open can leave it **half-written**.

Without `with`, you would write:

```python
f = open("report.txt", "w")
f.write("...")
f.close()          # never reached if write() fails
```

`with` guarantees the close, error or not. Older code, and some AI-generated code, still uses `open` … `close`: read it, but write `with`.

PART 2

Text in, numbers out

A CSV file, taken apart by hand, then in one line with pandas.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

text → numbers

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Converting each field

Line 12 converts the two numeric fields (session 2): `int("4")` is 4, `float("102.30")` is 102.3. The name stays text.

Each converted row becomes a **tuple** `(name, shares, price)`, added to the list `rows`:

```
[("Apple", 10, 190.5), ("Nestlé", 4, 102.3), ("Vanguard", 2, 512.0)]
```

A list of tuples: data we can loop over (session 3), from a file instead of the keyboard.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip(
10             ticker, name, shares, price = parts
11             rows.append((name, int(shares), float(price)))
12
13
14
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

shares is "" for MSFT

```
$ python report.py
Traceback (most recent call last):
  File "report.py", line 26, in <module>
    rows = load(Path("ledger.csv"))
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "report.py", line 11, in load
    rows.append((name, int(shares), float(price)))
                       ^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: ''
```

The bad row, without protection

Lines 11–14 lose their `try` and `except`. For MSFT, `shares` is the **empty string** `""`, and `int("")` refuses:

```
ValueError: invalid literal for int() with base 10: ''
```

The whole program stops at the **fourth** row: no report at all, because of one missing number. Real files **will** contain rows like this. Part 3 decides what to do about them.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = ""
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             try:
12                 rows.append((name, int(shares), float(price)))
13             except ValueError:
14                 print(f"skipped {ticker}: bad number")
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

header not read

```
$ python report.py
skipped ticker: bad number
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

the header, read as data

Forgot to skip the header

Line 7 no longer reads the header line, so the loop treats `ticker,name,shares,price` as a **position**. `int("shares")` fails, and the `except` skips it: **"skipped ticker"**.

Here the protection hides the mistake **almost** completely: the report is right. With a header of numbers (years, say), the header would have been counted as a position.

Rule: **know which line is the header**, and skip it on purpose.

report.py **CHANGED ON THIS SLIDE**

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(";")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

the wrong separator

```
$ python report.py
Traceback (most recent call last):
  File "report.py", line 26, in <module>
    rows = load(Path("ledger.csv"))
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "report.py", line 10, in load
    ticker, name, shares, price = parts
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 4, got 1)
```

The wrong separator

Line 9 now splits at `;`. No line contains one, so each "split" gives a list of **one** field, and line 10 cannot unpack it into four:

`ValueError: not enough values to unpack (expected 4, got 1)`

Files from European software often **do** use `;`, because the comma is the decimal separator: `102,30`. Look at the raw file before you read it.

frame.py

```python
# frame.py — the same ledger, the pandas way
import pandas as pd

df = pd.read_csv("ledger.csv")
print(df)
print(df["shares"].dtype, df["shares"].isna().sum())
df = df.dropna()
df["value"] = df["shares"] * df["price"]
print(df["value"].sum())
df.to_csv("report.csv", index=False)
print(open("report.csv").read())
```

the whole file, one line

```
$ python frame.py
  ticker       name  shares  price
0   AAPL      Apple    10.0  190.5
1   NESN     Nestlé     4.0  102.3
2    VOO   Vanguard     2.0  512.0
3   MSFT  Microsoft     NaN  410.0
float64 1
3338.2
ticker,name,shares,price,value
AAPL,Apple,10.0,190.5,1905.0
NESN,Nestlé,4.0,102.3,409.2
VOO,Vanguard,2.0,512.0,1024.0
```

the missing shares

The same file, with pandas

On the left, a new script. `pd.read_csv` does everything of lines 5–15 in **one line**: opens the file, reads the header, splits, converts.

The missing MSFT shares become `NaN`, "not a number": pandas' way of marking a **missing value**. No crash: the row is kept, with a hole in it.

Session 5 said pandas is the spreadsheet of Python. This is where it earns that name.

frame.py

```python
# frame.py — the same ledger, the pandas way
import pandas as pd

df = pd.read_csv("ledger.csv")
print(df)
print(df["shares"].dtype, df["shares"].isna().sum())
df = df.dropna()
df["value"] = df["shares"] * df["price"]
print(df["value"].sum())
df.to_csv("report.csv", index=False)
print(open("report.csv").read())
```

```
$ python frame.py
  ticker       name  shares  price
0   AAPL      Apple    10.0  190.5
1   NESN     Nestlé     4.0  102.3
2    VOO   Vanguard     2.0  512.0
3   MSFT  Microsoft     NaN  410.0
float64 1
3338.2
ticker,name,shares,price,value
AAPL,Apple,10.0,190.5,1905.0
NESN,Nestlé,4.0,102.3,409.2
VOO,Vanguard,2.0,512.0,1024.0
```

`read_csv` has options for every messy file

Problem	Option
fields separated by `;`	`sep=";"`
garbled accents	`encoding="latin-1"` (or the right one)
decimal comma, `102,30`	`decimal=","`
no header line	`header=None, names=[...]`
"n/a", "-" meaning missing	`na_values=["n/a", "-"]`
only some columns	`usecols=["ticker", "shares"]`

Every problem of Part 2 has an option. You do not memorise them: you **recognise** the problem, then look the option up.

frame.py

```python
# frame.py — the same ledger, the pandas way
import pandas as pd

df = pd.read_csv("ledger.csv")
print(df)
print(df["shares"].dtype, df["shares"].isna().sum())
df = df.dropna()
df["value"] = df["shares"] * df["price"]
print(df["value"].sum())
df.to_csv("report.csv", index=False)
print(open("report.csv").read())
```

the column's type

```
$ python frame.py
  ticker       name  shares  price
0   AAPL      Apple    10.0  190.5
1   NESN     Nestlé     4.0  102.3
2    VOO   Vanguard     2.0  512.0
3   MSFT  Microsoft     NaN  410.0
float64 1
3338.2
ticker,name,shares,price,value
AAPL,Apple,10.0,190.5,1905.0
NESN,Nestlé,4.0,102.3,409.2
VOO,Vanguard,2.0,512.0,1024.0
```

floats, one missing

⚠ WARNING

One missing value turns a column into floats

Look at output lines 2–5: the shares print as `10.0`, `4.0`, `2.0`. Line 6 confirms it: the column's type is `float64`, with 1 missing value.

`NaN` is itself a float, so a column with one gap cannot stay `int`. Your whole-number column changed type because of **one empty cell**, silently.

When a column looks "wrong", check its **dtype** and count its **missing values** (`.isna().sum()`), before anything else.

frame.py

```python
# frame.py — the same ledger, the pandas way
import pandas as pd

df = pd.read_csv("ledger.csv")
print(df)
print(df["shares"].dtype, df["shares"].isna().sum())
df = df.dropna()
df["value"] = df["shares"] * df["price"]
print(df["value"].sum())
df.to_csv("report.csv", index=False)
print(open("report.csv").read())
```

drop rows with gaps

write a file, one line

```
$ python frame.py
  ticker       name  shares  price
0   AAPL      Apple    10.0  190.5
1   NESN     Nestlé     4.0  102.3
2    VOO   Vanguard     2.0  512.0
3   MSFT  Microsoft     NaN  410.0
float64 1
3338.2
ticker,name,shares,price,value
AAPL,Apple,10.0,190.5,1905.0
NESN,Nestlé,4.0,102.3,409.2
VOO,Vanguard,2.0,512.0,1024.0
```

Dropping gaps, writing a file

- `df.dropna()` removes every row with a missing value: MSFT is gone. Line 9's total is **3338.2** again.
- `df.to_csv("report.csv", index=False)` writes the table to a new CSV file. `index=False` leaves out the row labels.

Output lines 8–11 show `report.csv`, read back. Plain text again, ready for Excel, a colleague, or the next program.

`dropna` is **one decision** about missing data. Deleting a row is not always right: a portfolio report that silently loses a position is wrong too.

frame.py

```python
# frame.py — the same ledger, the pandas way
import pandas as pd

df = pd.read_csv("ledger.csv")
print(df)
print(df["shares"].dtype, df["shares"].isna().sum())
df = df.dropna()
df["value"] = df["shares"] * df["price"]
print(df["value"].sum())
df.to_csv("report.csv", index=False)
print(open("report.csv").read())
```

```
$ python frame.py
  ticker       name  shares  price
0   AAPL      Apple    10.0  190.5
1   NESN     Nestlé     4.0  102.3
2    VOO   Vanguard     2.0  512.0
3   MSFT  Microsoft     NaN  410.0
float64 1
3338.2
ticker,name,shares,price,value
AAPL,Apple,10.0,190.5,1905.0
NESN,Nestlé,4.0,102.3,409.2
VOO,Vanguard,2.0,512.0,1024.0
```

By hand, or with pandas?

	By hand (`report.py`)	pandas (`frame.py`)
Lines of code	about 10	1
A missing value	a crash, unless you handle it	`NaN`, kept
You control	every step	the options: `sep=`, `encoding=`, …
Best for	odd formats, learning what a CSV is	almost all real work

Knowing the manual way is what lets you **read** `read_csv`'s options: `pd.read_csv("x.csv", sep=";", encoding="latin-1")` answers the problems of the last slides.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

+ GOOD TO KNOW

The `csv` module, and other formats

Python's standard library has a `csv` module, between the two: it splits lines correctly even when a field contains a comma inside quotes, `"Nestlé, S.A."`, which `.split(",")` gets wrong.

Other formats you will meet, each with one line in pandas:

Format	Read with
Excel `.xlsx`	`pd.read_excel("file.xlsx")`
JSON (session 5)	`pd.read_json(...)`, or the `json` module
Parquet: large, typed data	`pd.read_parquet(...)`

PART 3

Errors as tools

Catch what you expect, check what must be true, and let the rest crash.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

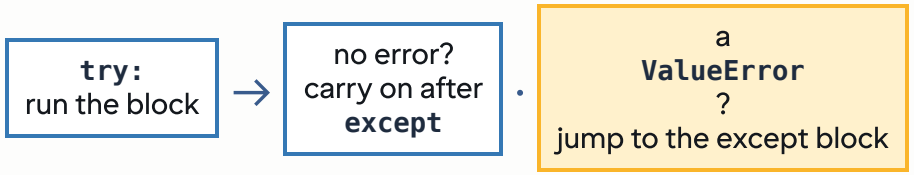
try this block

if it raises ValueError…

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

…do this instead

`try` / `except`: plan B for one kind of error



For AAPL, NESN and VOO, `int` and `float` succeed: the row is added, the except block is skipped. For MSFT, `int("")` raises a `ValueError`: Python **abandons** line 12 and runs line 14 instead.

The program **survives** the bad row, and **tells you** about it. Both matter.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

The `try` block, row by row

Row	`shares`	`int(shares)`	What runs
AAPL	`"10"`	`10`	line 12: row added
NESN	`"4"`	`4`	line 12: row added
VOO	`"2"`	`2`	line 12: row added
MSFT	`""`	`ValueError`	**line 14: "skipped MSFT"**

The loop goes on after the `except` block: **one** bad row costs **one** row, not the whole program. That is why the `try` sits **inside** the loop, around the conversion only.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

only this kind

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Name the error you expect

`except ValueError:` catches **only** `ValueError`: the error a bad number gives. Any other error still crashes, **loudly**, as it should.

Expected problem	Its error
text that is not a number	`ValueError`
a missing dict key	`KeyError`
a missing file	`FileNotFoundError`
dividing by zero	`ZeroDivisionError`

Catch the problems you **understand** and have a plan for. Everything else is a bug that should stop the program.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Keep the `try` block small

Put **only** the line that may fail inside `try`, not the whole program:

```python
try:
    rows = load(path)
    total = write_report(rows, out)
    print(total)
except ValueError:
    ...
```

```python
try:
    value = int(shares)
except ValueError:
    ...
```

The left version catches a `ValueError` from **anywhere** in three lines, including bugs you never thought of, and you cannot tell which line failed. The right version catches exactly one known problem. Lines 11–14 follow the right-hand pattern.

report.py **CHANGED ON THIS SLIDE**

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), flot(price)))
            except:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

a typo

catches everything

```
$ python report.py
skipped AAPL: bad number
skipped NESN: bad number
skipped VOO: bad number
skipped MSFT: bad number
Traceback (most recent call last):
  File "report.py", line 28, in <module>
    assert total > 0, "an empty report?"
           ^^^^^^^^^
AssertionError: an empty report?
```

⚠ WARNING

`except:` alone hides your bugs

Two changes: a typo, `flot` instead of `float`, and a bare `except:` that catches **every** error.

The typo raises a `NameError` on **every** row, and `except:` swallows it each time: all four rows "skipped", an **empty** report. Only the `assert` on line 28 stops it. Without it: a report that says nothing, no error.

AI assistants love `except Exception: pass`. It turns crashes, which tell you the truth, into **silent lies**. Name the error, or do not catch it.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             if shares == "":        # check first
12                 raise ValueError(f"no shares: {line.strip()}")
13             rows.append((name, int(shares), float(price)))
14
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

```
$ python report.py
Traceback (most recent call last):
  File "report.py", line 26, in <module>
    rows = load(Path("ledger.csv"))
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "report.py", line 12, in load
    raise ValueError(f"no shares: {line.strip()}")
ValueError: no shares: MSFT,Microsoft,,410.00
```

`raise`: stop, with a clear message

Sometimes skipping is the wrong plan: a report that silently loses a position may be worse than none. Lines 11–12 now **check** the field and **raise** our own error when it is empty.

`raise ValueError("…")` stops the program like any error, with **your** message on the last line: `no shares: MSFT,Microsoft,,410.00`. It names the row, so the next person knows exactly what to fix: the **data**, not the code.

Skip, or stop? That is a decision about the **data**, not about Python. Make it on purpose.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

must be True

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

`assert`: a check that must pass

`assert condition, "message"` does nothing when the condition is `True`. When it is `False`, the program stops with an `AssertionError` and your message.

Line 28 checks a fact we are sure of: a real portfolio has a **positive** total. Two slides ago, that one line caught a whole class of bugs.

Asserts are **tripwires** for things that "cannot happen". They cost one line and catch lies (failure kind 3) that would otherwise run silently.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Crash, skip, raise, or assert?

Situation	Do
a problem you did not expect	let it **crash**: the traceback is information
bad input you expect, and one bad row is harmless	`try` / `except`: **skip**, and say so
bad input that makes the result meaningless	**raise**, with a message about the data
something that must be true if the code is right	**assert** it

Never: catch everything and continue quietly.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

+ GOOD TO KNOW

`else`, `finally`, and error classes

A `try` can have two more blocks. You will read them more than write them:

```python
try:
    value = int(shares)
except ValueError:
    print("bad number")
else:
    print("only if no error")
finally:
    print("always, error or not")
```

Errors are **classes** (session 4): `FileNotFoundError` is a kind of `OSError`, and nearly all are kinds of `Exception`. `except OSError:` also catches a missing file. That is why `except Exception:` catches almost everything: too much.

PART 4

Tracebacks through functions and libraries

When a function calls a function, the error report shows the whole path.

report.py **CHANGED ON THIS SLIDE**

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledgr.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

frame 2: the failure

frame 1: the call

```
$ python report.py
Traceback (most recent call last):
  File "report.py", line 26, in <module>
    rows = load(Path("ledgr.csv"))
           ^^^^^^^^^^^^^^^^^^^^^^
  File "report.py", line 6, in load
    with open(path, encoding="utf-8") as f:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'ledgr.csv'
```

Two frames: a call, then a failure

> **frame 1** · line 26, in <module> · the top level **called** `load`
>
> > **frame 2** · line 6, in load · `open` **failed**
> >
> > **FileNotFoundError: … 'ledgr.csv'** · ***what went wrong***

"Most recent call last" (session 2): read **downwards** from the first call to the line that failed. Each `File …, line …, in …` is one **frame**: one function that was running.

The failure is in frame 2, but the **cause** is in frame 1: the name passed on line 26.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip(
10             ticker, name, shares, price = parts
11             rows.append((name, int(shares), float(price)))
12
13
14
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

fails here

called from here

```
$ python report.py
Traceback (most recent call last):
  File "report.py", line 26, in <module>
    rows = load(Path("ledger.csv"))
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "report.py", line 11, in load
    rows.append((name, int(shares), float(price)))
                       ^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: ''
```

The line that crashed is not the mistake

Again without the `try`: the traceback points at **line 11**, inside `load`, in the call from **line 26**.

But there is nothing wrong with line 11, nor with line 26. The problem is in the **data**: an empty field in `ledger.csv`.

A traceback shows **where the problem was noticed**. The cause may be above it in the frames, in an earlier line, or **outside the code** altogether. Read the last line, then ask: where did this **value** come from?

frame.py CHANGED ON THIS SLIDE

```python
# frame.py — the same ledger, the pandas way
import pandas as pd

df = pd.read_csv("ledgr.csv")
print(df)
print(df["shares"].dtype, df["shares"].isna().sum())
df = df.dropna()
df["value"] = df["shares"] * df["price"]
print(df["value"].sum())
df.to_csv("report.csv", index=False)
print(open("report.csv").read())
```

your line

```
$ python frame.py
Traceback (most recent call last):
  File "frame.py", line 4, in <module>
    df = pd.read_csv("ledgr.csv")
         ^^^^^^^^^^^^^^^^^^^^^^^^
  File ".../site-packages/pandas/io/parsers/readers.py", line 872, in
read_csv
    return _read(filepath_or_buffer, kwds)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File ".../site-packages/pandas/io/parsers/readers.py", line 300, in _read
    parser = TextFileReader(filepath_or_buffer, **kwds)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File ".../site-packages/pandas/io/parsers/readers.py", line 1643, in
__init__
    self._engine = self._make_engine(f, self.engine)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File ".../site-packages/pandas/io/parsers/readers.py", line 1907, in
_make_engine
    self.handles = get_handle(
                   ^^^^^^^^^^^
  File ".../site-packages/pandas/io/common.py", line 930, in get_handle
    handle = open(
             ^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'ledgr.csv'
```

A traceback from inside a library

The same typo in `frame.py`. Now the traceback runs **deep into pandas' own files**, `.../site-packages/pandas/...`, before the last line.

Those frames are pandas' code, calling its own functions. The mistake is almost never there. Look for:

1. the **last line**: `FileNotFoundError: … 'ledgr.csv'`;
2. the frame from **your own file**, `frame.py`, near the top: **line 4**.

Skip the library frames in between. Real pandas tracebacks can have twenty of them.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

REMINDER

Reading any traceback (sessions 2 and 6)

Step	Look at	Ask
1	the **last line**	what kind of error, and its message?
2	the **frames from your files**	which of my lines was running?
3	the `^^^^` under the code	which piece of that line?
4	the **values** on that line	where did they come from? (a frame above, or the data)

Error types met so far: `NameError`, `TypeError`, `ValueError`, `KeyError`, `IndexError`, `AttributeError`, `FileNotFoundError`, `AssertionError`.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Errors you will meet with files

Error	Usually means
`FileNotFoundError`	wrong name, or wrong working directory
`UnicodeDecodeError`	the file is not in the encoding you asked for
`PermissionError`	no right to read or write there, or the file is open in Excel
`IsADirectoryError`	you gave a folder, not a file
`ValueError` when unpacking	a line with the wrong number of fields

Wrong encodings do not always crash: `latin-1` **never** fails, it silently garbles (the `NestlÃ©` slide). `ascii` or `utf-8` on the wrong file raise `UnicodeDecodeError`.

PART 5

Debugging, as a method

Finding a bug is not luck. It is a procedure.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

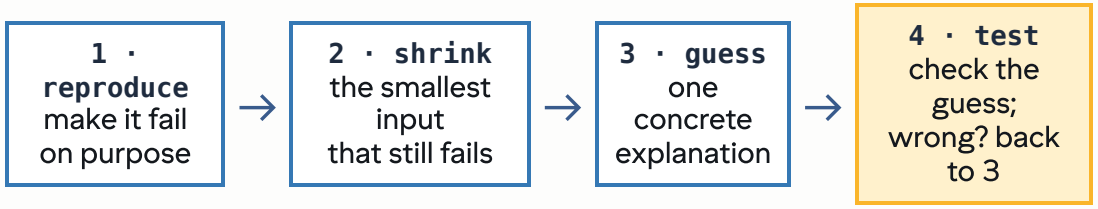
def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Four steps



- **Reproduce**: a bug you cannot trigger again, you cannot check you fixed.
- **Shrink**: one bad row, not 10,000. The fewer lines, the fewer suspects.
- **Guess**, then **test**: change **one thing** at a time, or you will not know what worked.

Guessing and changing code at random, or asking an AI to "fix it" repeatedly, is not debugging. It is hoping.

report.py **CHANGED ON THIS SLIDE**

```
1  # report.py — ledger.csv in, report.txt out
2  from pathlib import Path
3
4  def load(path):
5      rows = []
6      with open(path, encoding="utf-8") as f:
7          header = f.readline()
8          for line in f:
9              parts = line.strip().split(",")
10             ticker, name, shares, price = parts
11             try:
12                 rows.append((name, int(shares), float(price)))
13             except ValueError:
14                 print(f"skipped {ticker}: {shares=}")
15     return rows
16
17 def write_report(rows, path):
18     total = 0.0
19     with open(path, "w", encoding="utf-8") as out:
20         for name, shares, price in rows:
21             out.write(f"{name:<9}{shares * price:>10,.2f}\n")
22             total += shares * price
23         out.write(f"{'total':<9}{total:>10,.2f}\n")
24     return total
25
26 rows = load(Path("ledger.csv"))
27 total = write_report(rows, Path("report.txt"))
28 assert total > 0, "an empty report?"
29 print(len(rows), "positions, total", round(total, 2))
30 print(Path("report.txt").read_text())
```

name and value

```
$ python report.py
skipped MSFT: shares=''
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

an empty string!

Look inside: `print(f"{x=}")`

Line 14 now prints the value that caused the skip, with session 5's `=` trick: `shares=''`. The quotes show it is an **empty string**, not a space, not zero.

The fastest debugging tool is still `print`: put it **just before** the line that fails, and print the values that line uses. Then read them **with** `repr` when they are text: invisible characters become visible.

Remove the prints when you are done.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20   4 × 102.30 = 409.20 ✓
Vanguard   1,024.00
total      3,338.20
```

Check one number by hand

A program that runs is not a program that is right (session 1). Pick **one** number from the output and compute it yourself:

- Nestlé: 4 shares × 102.30 = **409.20**. ✓
- The total: 1,905 + 409.20 + 1,024 = **3,338.20**. ✓

If one number is right, the formula is **probably** right. If it is wrong, you have found a bug that no error message would ever show. Do this **every time** a program gives you a number that matters.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

+ GOOD TO KNOW

Asking an AI for help, well

A chatbot cannot see your screen or your files (session 1). Give it what a human expert would need:

1. the **full traceback**, copied, not summarised;
2. the **code** of the functions in the frames from your files;
3. a **sample** of the data: the header and a few rows, including the bad one;
4. what you **expected**, and what happened instead.

Then **check** its answer: run it, read it, and look for a new `except Exception:`. The fix you understand is worth more than the fix that makes the error disappear.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

+ GOOD TO KNOW

Explain it to a rubber duck

Programmers have an old trick: explain the code, **line by line, out loud**, to someone who knows nothing, even a rubber duck on the desk.

Saying "this line reads the header" forces you to notice when it does **not**. Surprisingly often, the bug appears halfway through your own sentence.

It works on paper too: trace the values by hand, as in session 1. VS Code also has a **debugger** that pauses a program line by line; you will not need it in this course, but it is there.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

↺ REMINDER

The bugs we have met (sessions 1–6)

Bug	Seen in	Catch it by
text where a number was meant: `"4" * 2`	session 2	`type()`, `repr()`
float rounding, `==` on money	session 2	a tolerance
boundaries: `>` vs `>=`, off by one	session 3	test the edge case
two names, one list	sessions 3–4	`.copy()`, read the code
wrong axis, wrong broadcasting	session 5	shapes, one number by hand
missing data made up or hidden	session 6	say it, `raise`, `assert`

Almost none of them crash. That is why checking **results**, not just "no error", is the job.

PART 6

The complete program

Load, check, compute, write: session 1's promise, kept.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

1. load

2. compute and write

3. check

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

The shape of every data program



Lines 26–30 read like the diagram: the top level **tells the story**, the functions hold the details (session 4).

Every data program you will write in Part 2, and most you will read, has this shape. Only the size of each box changes.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Why two functions?

- `load(path)` knows about **the file format**: commas, headers, bad rows. Nothing about reports.
- `write_report(rows, path)` knows about **the report**: columns, totals. Nothing about CSV.

Change the file to Excel, and only `load` changes. Want a different report, and only `write_report` changes.

Each function has **one job**, and passes plain data (a list of tuples) to the next. That is what makes a program easy to test, fix, and hand to someone else.

report.py **CHANGED ON THIS SLIDE**

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                rows.append((name, 0, float(price)))
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

a classmate's version

```
$ python report.py
4 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
Microsoft      0.00
total      3,338.20
```

is this right?

Spot the lie

A classmate does not like losing a position, so the except block now keeps MSFT with **0 shares**.

It runs, with no message. The report lists four positions, including Microsoft. The total is still 3,338.20.

Is this report honest? Which kind of failure is it?

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

honest: said out loud

The lie, explained

The file does not say MSFT has **zero** shares. It says nothing: the number is **missing**. Writing `0` invents data, and the report now claims "Microsoft: 0.00", which nobody knows.

Failure kind 3: it runs and lies, and it also **hid the problem**: no message, so nobody will fix the file.

Session 2: `None` is not `0`. Missing is not zero. **Skip and say so, raise, or ask**, but never quietly make a number up.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Read it again

The same 30 lines as at the start. Every line is now yours:

- `with open(...)` for reading (line 6) and for writing (line 19), with an encoding;
- `readline`, a loop over lines, `strip` and `split`;
- `try` / `except ValueError` for the bad row, and `assert` as a tripwire;
- two functions with one job each, and a top level that tells the story;
- `Path` objects from `pathlib`.

report.py

```python
# report.py — ledger.csv in, report.txt out
from pathlib import Path

def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        header = f.readline()
        for line in f:
            parts = line.strip().split(",")
            ticker, name, shares, price = parts
            try:
                rows.append((name, int(shares), float(price)))
            except ValueError:
                print(f"skipped {ticker}: bad number")
    return rows

def write_report(rows, path):
    total = 0.0
    with open(path, "w", encoding="utf-8") as out:
        for name, shares, price in rows:
            out.write(f"{name:<9}{shares * price:>10,.2f}\n")
            total += shares * price
        out.write(f"{'total':<9}{total:>10,.2f}\n")
    return total

rows = load(Path("ledger.csv"))
total = write_report(rows, Path("report.txt"))
assert total > 0, "an empty report?"
print(len(rows), "positions, total", round(total, 2))
print(Path("report.txt").read_text())
```

```
$ python report.py
skipped MSFT: bad number
3 positions, total 3338.2
Apple      1,905.00
Nestlé       409.20
Vanguard   1,024.00
total      3,338.20
```

Check yourself

On paper first, then with your neighbour:

1. What does `"  NESN,4\n".strip().split(",")` give?
2. You open `report.txt` with `"w"` twice. How many reports does it contain?
3. A CSV has `102,30` as a price. What went wrong, and what `read_csv` option fixes it?
4. Name one error you should catch with `except`, and one you should let crash.
5. A traceback has five frames: which one do you read first, and which do you fix?

What you can now do

- Open, read and write text files with `with open(...)`, and choose the mode and encoding.
- Turn the lines of a CSV into numbers, by hand and with `pd.read_csv`.
- Recognise missing values, and decide on purpose whether to skip, raise, or keep them.
- Use `try` / `except` for the errors you expect, `raise` and `assert` for your own checks.
- Read a traceback through functions and libraries, and find your own line.
- Debug with a method: reproduce, shrink, guess, test, and check one number by hand.